



Programmation orientée objet : Java

POO: PROGRAMMATION ORIENTÉE OBJET

INTRODUCTION AU POO

CONSTATS

- Il y a beaucoup de manières à écrire un programme qui effectue une tâche spécifiée.
- La manière de programmation dépend du langage utilisé.
- Le langage utilisé dépend de la manière de programmation.

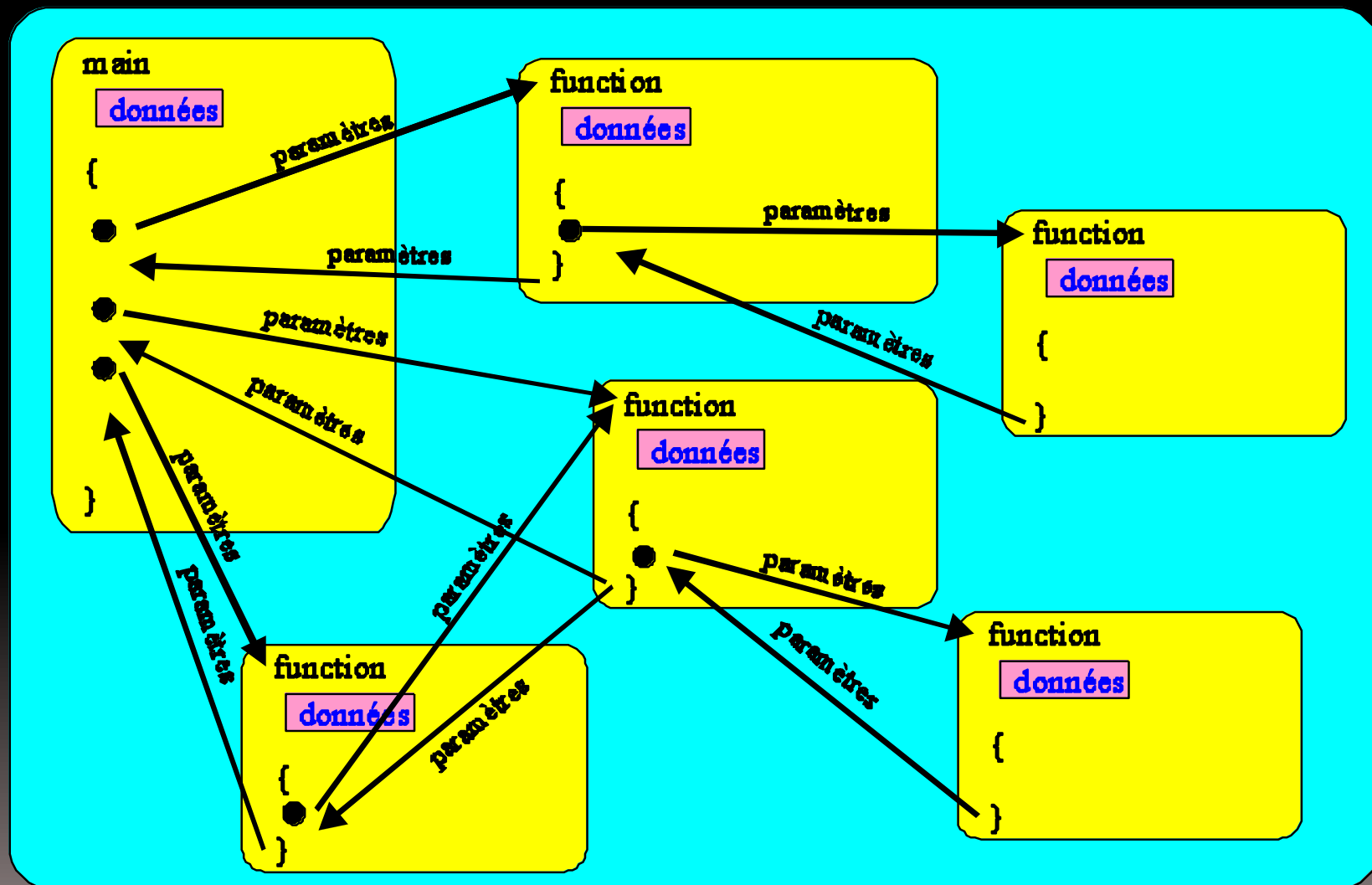
PARADIGMES DE PROGRAMMATION

- programmation procédurale : P.P. (Pascal, C, etc.)
- programmation orientée objet : P.O.O. (C++, Java, ...)
- etc.

PROGRAMMATION PROCÉDURALE (RAPPEL DE C)

- Le programme est composé des fonctions
- Les données (variables) sont créées à l'intérieure des fonction ou bien passées comme paramètres
- Il y a un programme principal

UN PROGRAMME EN C



LIMITATIONS

- Il n'y a pas de méthode ou de cadre pour bien organiser les fonctions.
- Les modifications d'une fonction entraînent d'autres modifications dans autre fonctions, etc. – la portée d'une modification est trop grand et difficile à gérer.
- Redondance dans le code (la même chose est codé plusieurs fois)
- Propagation des erreurs – débogage difficile

EST-CE QU'IL FAUT OUBLIER LE C?



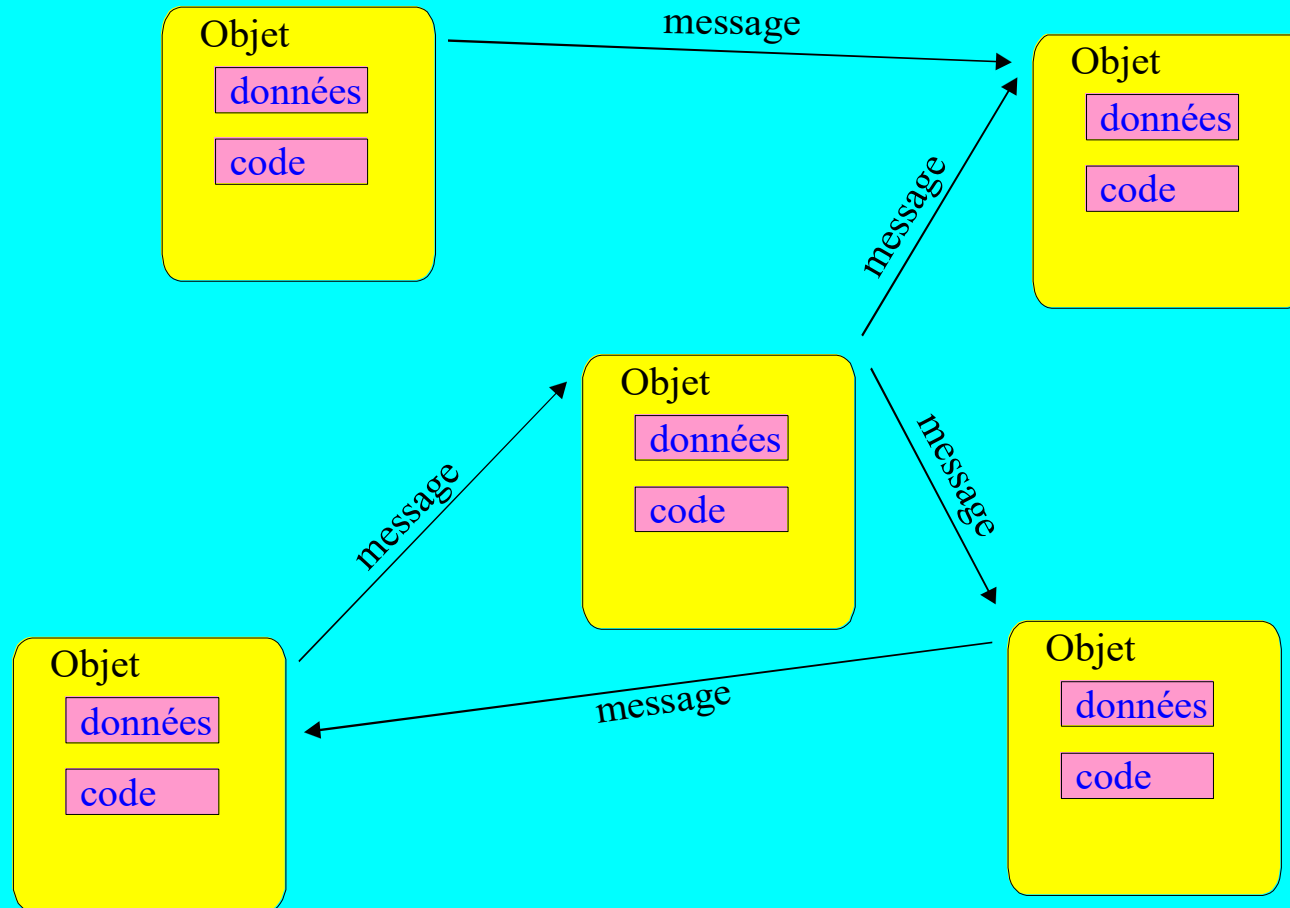
NON!

Vous allez avoir le
choix parmi
plusieurs
méthodes de
programmation!

PROGRAMMATION ORIENTÉ OBJET

- Le POO est un paradigme et n'est pas un langage.
- Modélisation du domaine à l'aide des classes
- Définition des classes
- Création des instances
- Messages entre les objets (appel des méthodes)

UN PROGRAMME ORIENTÉ OBJET



INTÉRÊT DE LA POO

- Le code est plus sûr
- Les programmes sont plus clairs
- La maintenance des applications est facilitée
- Le code est facilement réutilisable
- Il est facile de créer de nouveaux algorithmes légèrement différents par clonage d'un algorithme existant
- Il est facile de faire évoluer des programmes

CLASSE ET OBJET

LES CARACTÉRISTIQUES DE L'OBJET

La 1ère étape consiste à déterminer

- les entités que l'on souhaite manipuler
- la description générique qui relie toutes ses entités.

EXEMPLE

- Prenons ces informations :
 - 21 Janvier 2018 (Examen)
 - 18 Novembre 1955 (l'indépendance du Maroc)
 - 01 aout 2019
- Ce sont des dates.
- Chaque date se caractérise par :
 - un *jour*
 - un *mois*
 - une *année*.

Les Dates

Date

- Jour
- Mois
- Année

Examen

- 21
- Janvier
- 2018

Une Date

- 1
- Aout
- 2019

l'indépendance

- 18
- Novembre
- 1955

CLASSES

- Le *Jour*, le *Mois* et l'*Année* sont les **attributs** d'une Date.
- Cet ensemble d'attributs est appelé une **Classe**.

OBJETS

- Le 21 Janvier 2018 et le 18 Novembre 1955 sont chacune des **instances** de la classe Date.
- Chacune de ces dates est appelée un **Objet**.

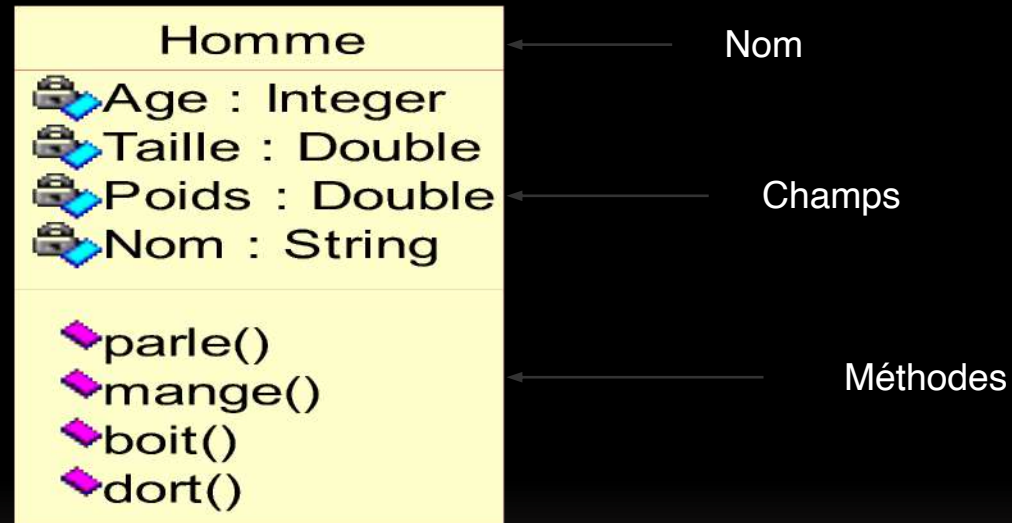
L'OBJET : DÉFINITION

- Qu'est-ce qu'un objet ?
 - Le monde qui nous entoure est composé d'objets
 - Ces objets ont tous deux caractéristiques
 - un état
 - un comportement
- Un objet informatique
 - maintient son état dans des variables (appelées champs)
 - implémente son comportement à l'aide de méthodes
- Cycle de vie
 - construction (en mémoire)
 - Utilisation (changements d'état par affectations, comportements par exécution de méthodes)
 - destruction

LA CLASSE : DÉFINITION

- **Classe** : description d'une famille d'objets ayant une même structure et un même comportement. Elle est caractérisée par :
 - Un nom
 - Une composante statique : des **champs** (ou **attributs**) nommés ayant une valeur. Ils caractérisent l'état des objets pendant l'exécution du programme
 - Une composante dynamique : des **méthodes** représentant le comportement des objets de cette classe. Elles manipulent les champs des objets et caractérisent les actions pouvant être effectuées par les objets.

LA CLASSE : REPRÉSENTATION GRAPHIQUE



Une classe représentée avec la notation UML

```
Class Point ← Nom de la Classe
{
    double x; // abscisse du point
    double y; // ordonnée du point ← Attributs
    // translate de point de (dx,dy)
    void translate (double dx, double dy) {
        x = x+dx; ← Méthodes
        y = y+dy;
    }
    // calcule la distance du point à l'origine
    double distance() {
        return Math.sqrt(x*x+y*y);
    }
}
```

21

L'INSTANCIATION

- Instanciation : concrétisation d'une classe en un objet « *concret* ».
 - Dans nos programmes Java nous allons définir des classes et *instancier* ces classes en des objets qui vont interagir. Le fonctionnement du programme résultera de l'interaction entre ces objets « *instanciés* ».
 - En Programmation Orientée Objet, on décrit des classes et l'application en elle-même va être constituée des objets instanciés, à partir de ces classes, qui vont communiquer et agir les uns sur les autres.

L'INSTANCIATION

- Instance
 - représentant physique d'une classe
 - obtenu par moulage du dictionnaire des variables et détenant les valeurs de ces variables.
 - Son comportement est défini par les méthodes de sa classe
 - Par abus de langage « instance » = « objet »
- Exemple :
 - si nous avons une classe voiture, alors votre voiture est une instance particulière de la classe voiture.

LES CONSTRUCTEURS

- L'appel de **new** pour créer un nouvel objet déclenche, dans l'ordre :
 - L'allocation mémoire nécessaire au stockage de ce nouvel objet et l'initialisation par défaut de ces attributs,
 - L'initialisation explicite des attributs, s'il y a lieu,
 - L'exécution d'un constructeur.
- Un **constructeur** est une méthode d'initialisation.

LES CONSTRUCTEURS

- Lorsque l'initialisation explicite n'est pas possible (par exemple lorsque la valeur initiale d'un attribut est demandée dynamiquement à l'utilisateur), il est possible de réaliser l'initialisation au travers d'un constructeur.
- Le constructeur est une méthode :
 - de même nom que la classe,
 - sans type de retour.
- Toute classe possède au moins un constructeur. Si le programmeur ne l'écrit pas, il en existe un par défaut, sans paramètres, de code vide.

LES CONSTRUCTEURS

Personne.java

```
public class Personne
{
    public String nom;
    public String prenom;
    public int age;
    public Personne(String unNom,
                     String unPrenom,
                     int unAge)
    {
        nom=unNom;
        prenom=unPrenom;
        age = unAge;
    }
}
```

Définition d'un Constructeur. Le constructeur par défaut (Personne())n'existe plus. Le code précédent occasionnera une erreur

```
public class Application
{
    public static void main(String args[])
    {
        Personne jean = new Personne()
        jean.setNom("Jean");
    }
}
```

Va donner une erreur à la compilation

LES CONSTRUCTEURS

- Pour une même classe, il peut y avoir plusieurs constructeurs, de signatures différentes (surcharge).
- L'appel de ces constructeurs est réalisé avec le **new** auquel on fait passer les paramètres.
 - **p1 = new Personne("Pierre", "Richard", 56);**
- Déclenchement du "bon" constructeur
 - Il se fait en fonction des paramètres passés lors de l'appel (nombre et types).
- Attention
 - Si le programmeur crée un constructeur (même si c'est un constructeur avec paramètres), le constructeur par défaut n'est plus disponible. Attention aux erreurs de compilation !

LES CONSTRUCTEURS

Personne.java

```
public class Personne
{
    public String nom;
    public String prenom;
    public int age;
    public Personne()
    {
        nom=null; prenom=null;
        age = 0;
    }
    public Personne(String unNom, String unPrenom, int unAge)
    {
        nom=unNom;
        prenom=unPrenom; age = unAge;
    }
}
```

Redéfinition d'un
Constructeur sans paramètres

On définit plusieurs constructeurs
qui se différencient uniquement
par leurs paramètres (on parle
de leur signature)

CLASSE ET OBJET EN JAVA

Du modèle à ...



... la classe Java et
de la classe à ...

```
class Personne
{
    String nom;
    int age;
    float salaire;
}
```

... des instances
de cette classe

```
Personne jean, pierre;
jean = new Personne ();
pierre = new Personne ();
```

L'opérateur d'instanciation en Java est **new** :

```
MaClasse monObjet = new MaClasse();
```

En fait, new va réserver l'espace mémoire nécessaire pour créer l'objet

« monObjet » de la classe « MaClasse »

Le new ressemble beaucoup au malloc du C

ACCÈS AUX ATTRIBUTS D'UN OBJET

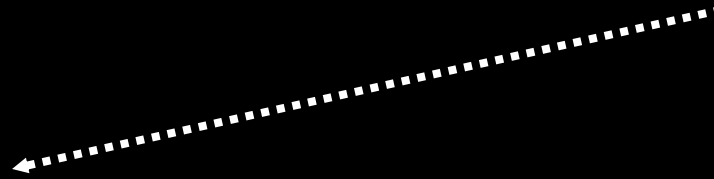
Personne.java

```
public class Personne
{
    public String nom;
    public String prenom;
    public int age;
    public void setNom(String unNom)
    {
        nom = unNom;
    }
    public String getNom()
    {
        return (nom);
    }
    ...
}
```

Personne

◆ nom : String
◆ prenom : String
◆ age : Integer

◆ getNom()
◆ setNom()



ACCÈS AUX ATTRIBUTS D'UN OBJET

Application.java

```
public class Application
{
    public static void main(String args[])
    {
        Personne jean = new Personne()
        jean.nom = "Jean" ;
        jean.prenom = "Pierre" ;
    }
}
```

Application.java

```
public class Application
{
    public static void main(String args[])
    {
        Personne jean = new Personne()
        jean.setNom("Jean") ;
        jean.setPrenom("Pierre") ;
    }
}
```

Remarque

Contrairement aux variables, les attributs d'une classe, s'ils ne sont pas initialisés, se voient affecter automatiquement une valeur par défaut.

Cette valeur vaut : **0** pour les variables numériques, **false** pour les booléens, et **null** pour les références.

NOTION DE MÉTHODES ET DE PARAMÈTRES

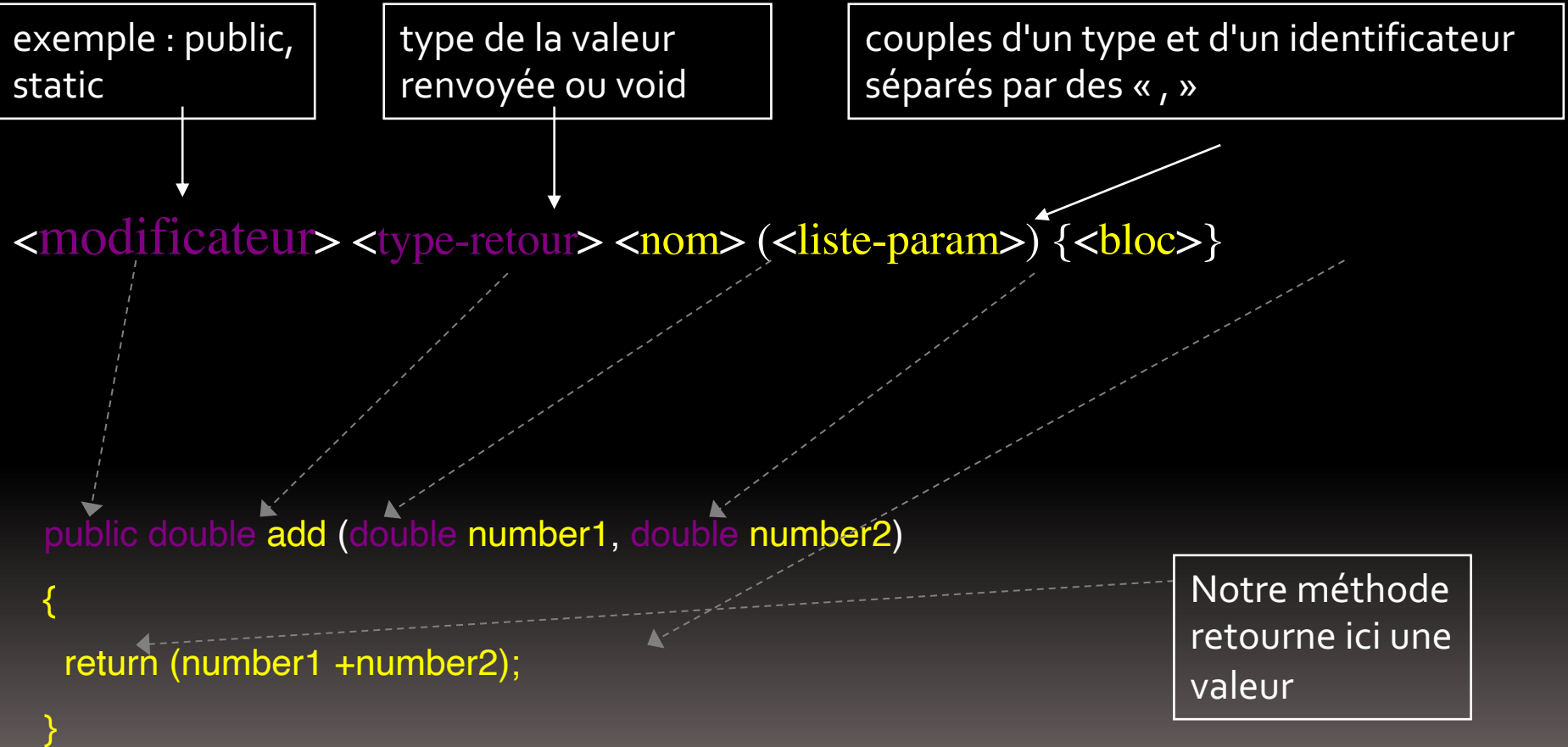
La notion de méthodes dans les langages objets :

- Proches de la notion de procédure ou de fonction dans les langages procéduraux.
- La méthode c'est avant tout le regroupement d'un ensemble d'instructions suffisamment générique pour pouvoir être réutilisées
- On peut passer des paramètres aux méthodes et ces dernières peuvent renvoyer des valeurs (grâce au mot clé **return**).

MODE DE PASSAGE DES PARAMÈTRES

- Java n'implémente qu'un seul mode de passage des paramètres à une méthode : le passage par valeur.
- Conséquences :
 - l'argument passé à une méthode ne peut être modifié,
 - si l'argument est une instance, c'est sa référence qui est passée par valeur. Ainsi, le contenu de l'objet peut être modifié, mais pas la référence elle-même. (On verra cela en détails)

NOTION DE MÉTHODES ET DE PARAMÈTRES



PORTÉE DES VARIABLES

- Les variables sont connues et ne sont connues qu'à l'intérieur du bloc dans lequel elles sont déclarées

```
public class Bidule
{
    int i, j, k;
    public void truc(int z)
    {
        int j,r;
        r = z;
    }
    public void machin()
    {
        k = r;
    }
}
```

Ce sont 2 variables différentes

k est connu au niveau de la méthode machin() car déclaré dans le bloc de la classe. Par contre r n'est pas défini pour machin(). On obtient une erreur à la compilation

PORTÉE DES VARIABLES

- En cas de conflit de nom entre des variables locales et des variables globales, c'est toujours la variable la plus locale qui est considérée comme étant la variable désignée par cette partie du programme

C'est le j défini en local
qui est utilisé dans
la méthode truc()

```
public class Bidule
{
    int i, k, j;
    public void truc(int z)
    {
        int j,r;
        j = z;
    }
}
```



L'AUTORÉFÉRENCE : THIS

- Le mot réservé **this**, utilisé dans une méthode, désigne la référence de l'instance courante.
- Il est utilisé principalement :
 - lorsqu'une référence à l'instance courante doit être passée en paramètre à une méthode,
 - pour lever une ambiguïté,
 - dans un constructeur, pour appeler un autre constructeur de la même classe.

L'AUTORÉFÉRENCE : THIS

```
class Personne
{
    public String nom;
    Personne (String nom)
    {
        this.nom=nom;
    }
}
```

Pour lever l'ambiguïté sur le mot « nom »
et déterminer si c'est le nom du paramètre
ou de l'attribut

```
public MaClasse(int a, int b) {...}
public MaClasse (int c)
{
    this(c,o);
}
public MaClasse ()
{
    this(10);
}
```

Appelle le constructeur
MaClasse(int a, int b)

Appelle le constructeur
MaClasse(int c)

L'ENCAPSULATION

- Notion d'encapsulation :
 - les données et les procédures qui les manipulent sont regroupées dans une même entité, l'objet.
 - Les détails d'implémentation sont cachés, le monde extérieur n'ayant accès aux données que par l'intermédiaire d'un ensemble d'opérations constituant l'**interface** de l'objet.
 - Le programmeur n'a pas à se soucier de la représentation physique des entités utilisées et peut raisonner en termes d'abstractions.

L'ENCAPSULATION

- Programmation dirigée par les données :
 - pour traiter une application, le programmeur commence par définir les classes d'objets appropriées, avec leurs opérations spécifiques.
 - chaque entité manipulée dans le programme est un représentant (ou instance) d'une de ces classes.
 - L'univers de l'application est par conséquent composé d'un ensemble d'objets qui détiennent, chacun pour sa part, les clés de leur comportement.

CONTRÔLE D'ACCÈS

- Chaque attribut et chaque méthode d'une classe peut être :
 - visible depuis les instances de toutes les classes d'une application. En d'autres termes, son nom peut être utilisé dans l'écriture d'une méthode de ces classes. Il est alors **public**.
 - visible uniquement depuis les instances de sa classe. En d'autres termes, son nom peut être utilisé uniquement dans l'écriture d'une méthode de sa classe. Il est alors **privé**.
- Les mots réservés sont :
 - **public**
 - **private**

CONTRÔLE D'ACCÈS

- En toute rigueur, il faudrait toujours que :
 - les attributs ne soient pas visibles,
 - Les attributs ne devraient pouvoir être lus ou modifiés que par l'intermédiaire de méthodes prenant en charge les vérifications et effets de bord éventuels.
 - seules les fonctionnalités de l'objet, destinées à être utilisées par d'autres objets soient visibles.
 - C'est la notion d'**encapsulation**
- Nous verrons dans la suite que l'on peut encore affiner le contrôle d'accès

Contrôle d'accès

```
public class Parallelogramme
{
    private int longueur = 0; // déclaration + initialisation explicite
    private int largeur = 0;   // déclaration + initialisation explicite
    public int profondeur = 0; // déclaration + initialisation explicite
    public void affiche ( )
    {
        System.out.println("Longueur= " + longueur + " Largeur = " + largeur +
                           " Profondeur = " + profondeur);
    }
}
```

```
public class ProgPpal
{
    public static void main(String args[])
    {
        Parallelogramme p1 = new Parallelogramme();
        p1.longueur = 5; // Invalide car l'attribut est privé
        p1.profondueur = 4; // OK
        p1.affiche( ); // OK
    }
}
```

VARIABLES DE CLASSE

- Il peut s'avérer nécessaire de définir un attribut dont la valeur soit partagée par toutes les instances d'une classe. On parle de variable de classe.
- Ces variables sont, de plus, stockées une seule fois, pour toutes les instances d'une classe.
- Mot réservé : **static**
- Accès :
 - depuis une méthode de la classe comme pour tout autre attribut,
 - via une instance de la classe,
 - à l'aide du nom de la classe.

VARIABLES DE CLASSE

```
public class UneClasse
{
    public static int compteur = 0;
    public UneClasse ()
    {
        compteur++;
    }
}

public class AutreClasse
{
    public void uneMethode()
    {
        int i = UneClasse.compteur;
    }
}
```

Variable de classe

Utilisation de la variable de classe
compteur dans le constructeur de
la classe

Utilisation de la variable de classe
compteur dans une autre classe

MÉTHODES DE CLASSE

- Il peut être nécessaire de disposer d'une méthode qui puisse être appelée sans instance de la classe. C'est une méthode de classe.
- On utilise là aussi le mot réservé **static**
- Puisqu'une méthode de classe peut être appelée sans même qu'il n'existe d'instance, une méthode de classe ne peut pas accéder à des attributs non statiques. **Elle ne peut accéder qu'à ses propres variables et à des variables de classe.**

MÉTHODES DE CLASSE

```
public class UneClasse
{
    public int unAttribut;
    public static void main(String args[])
    {
        unAttribut = 5; // Erreur de compilation
    }
}
```

La méthode main est une méthode de classe donc elle ne peut accéder à un attribut non lui-même attribut de classe



Exercice